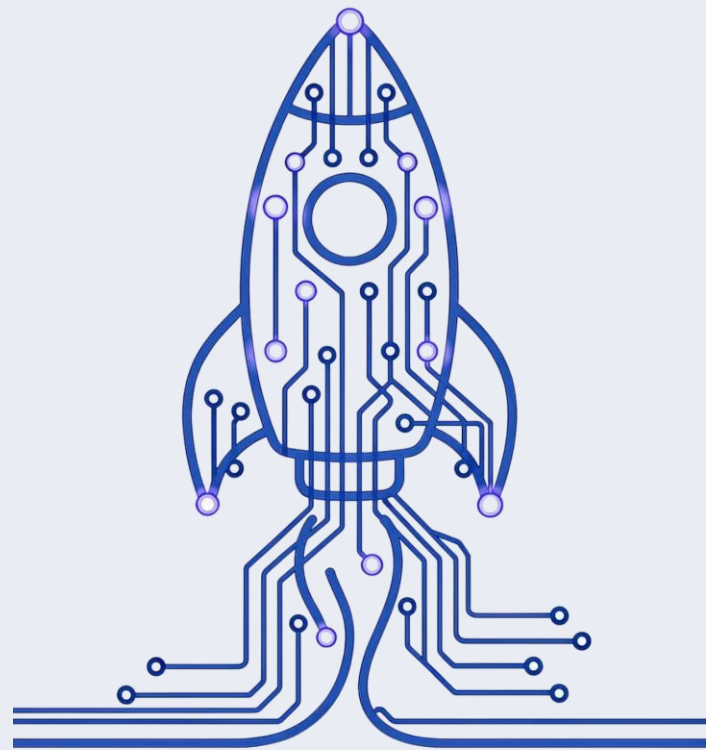




Introducción a la Computación 2

Estructuras Dinámicas de Datos en C++

Memoria dinámica, punteros, estructuras, listas enlazadas, TADs y matrices dispersas — los bloques fundamentales del software moderno.

¿Por qué estructuras dinámicas?

Los programas del mundo real no saben cuántos datos procesarán en tiempo de compilación. Las estructuras dinámicas permiten que el software **crezca, se adapte y libere recursos** según la demanda del sistema.



Bases de Datos

Tablas con millones de registros que crecen dinámicamente en tiempo de ejecución.



Redes Sociales

Grafos de usuarios y conexiones que se expanden con cada nuevo registro.



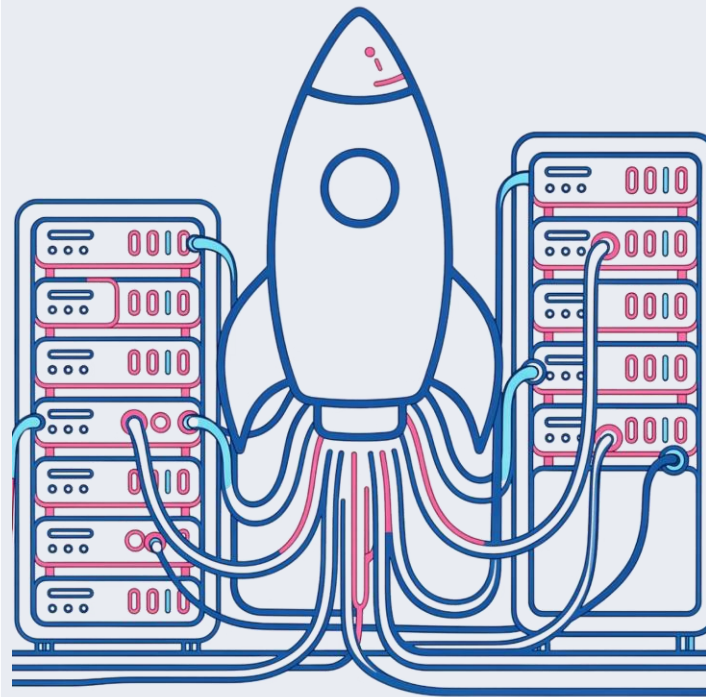
Inteligencia Artificial

Modelos que almacenan capas y matrices dispersas para optimizar el cómputo.



Videojuegos

Mundos abiertos con objetos que se crean y destruyen en tiempo real.



Memoria Dinámica

HEAP VS STACK

Conceptos clave

En C++, la memoria se divide en dos regiones principales. El **Stack** almacena variables locales con tamaño fijo conocido en compilación. El **Heap** permite solicitar bloques de memoria en tiempo de ejecución con `new` y liberarlos con `delete`.

```
// Asignación dinámica en C++
int* p = new int(42); // reserva en Heap
cout << *p;           // imprime 42
delete p;             // libera memoria
p = nullptr;          // evita dangling pointer

// Arreglo dinámico
int* arr = new int[10];
delete[] arr;          // libera arreglo
```

Stack

- Tamaño fijo en compilación
- Gestión automática (LIFO)
- Variables locales y parámetros

Heap

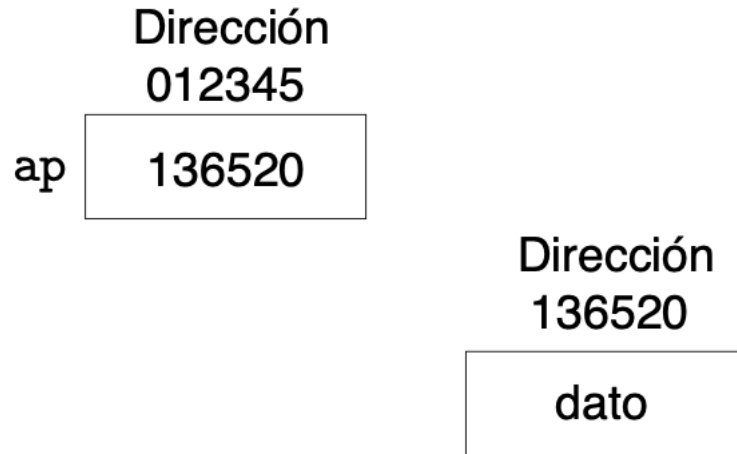
- Tamaño variable en ejecución
- Gestión manual con `new/delete`
- Persistencia entre funciones
- Riesgo de memory leaks



Siempre liberar con `delete` toda memoria reservada con `new` para evitar fugas de memoria (memory leaks).

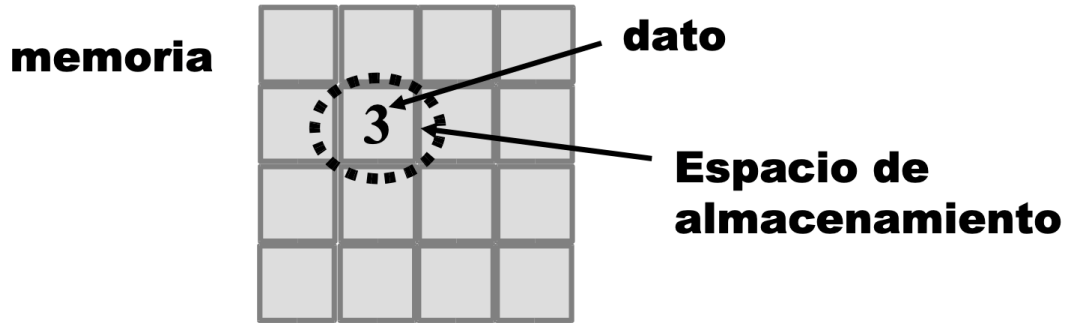
Variables punteros

Un puntero es una variable que sirve para señalar la posición de la memoria en que se encuentra otro dato almacenando como valor (la dirección de ese dato).



Variables punteros

Antes de continuar es importante recordar aspectos como los siguientes:



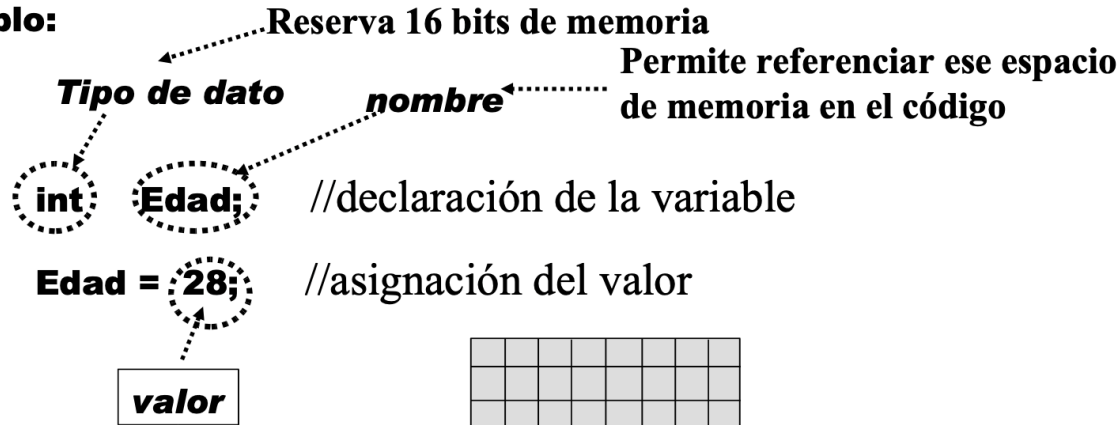
Toda variable tiene:

- **nombre (identificador)**,
- ***tipo de dato, que permite reservar un espacio de almacenamiento (dirección de memoria) y***
- **contenido**.

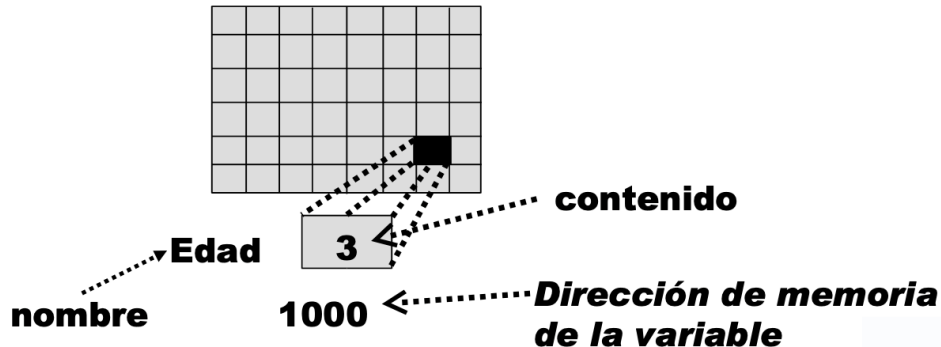
Punteros

El programador al realizar la declaración de la variable, reserva un espacio de almacenamiento en memoria, conoce el nombre, pero el Sistema Operativo (S.O.) es quien asigna la dirección de memoria.

Ejemplo:

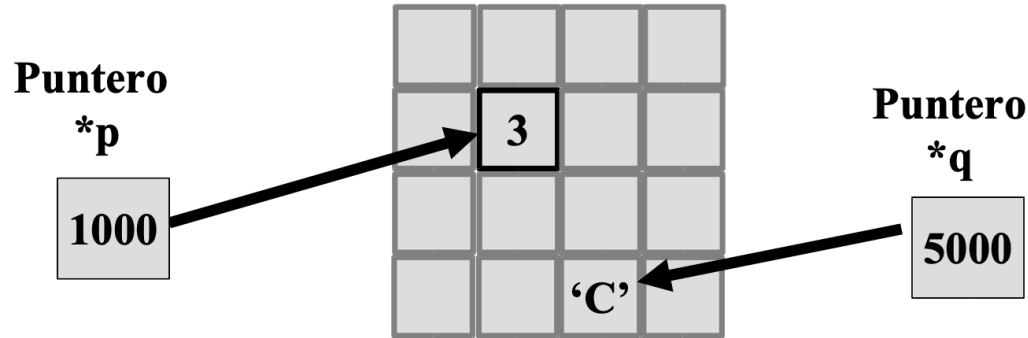


El S.O. encuentra un espacio libre de memoria y la asocia con el nombre de la variable.



Variables punteros

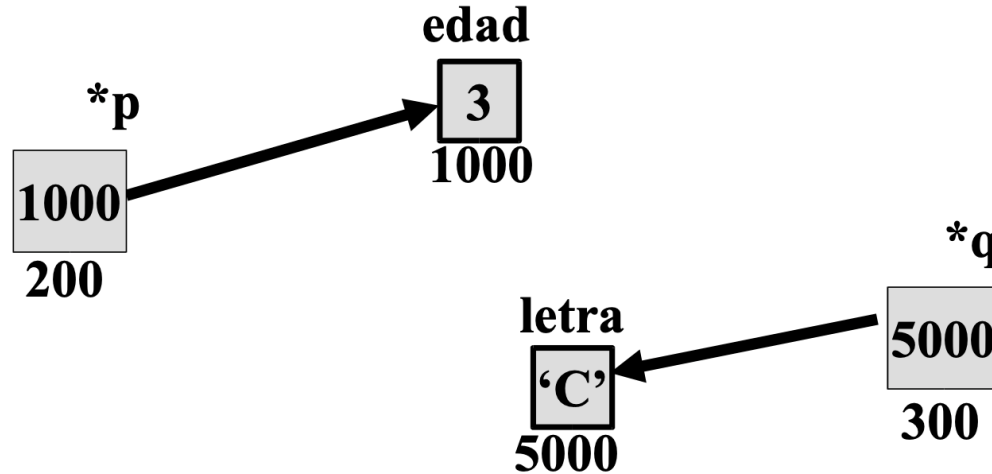
Un puntero puede apuntar a un objeto de cualquier tipo, incluyendo variables simples, arreglos, estructuras, funciones, etc.



Por esa razón su contenido es una dirección de memoria y no un valor cualquiera.

Variables punteros

Las variables tienen nombre, contenido y dirección de memoria.



Direcciones de memoria

Los datos en la memoria

Todo dato se almacena en memoria:
Varios bytes a partir de una dirección

```
int i = 5;
```

Dirección base

i

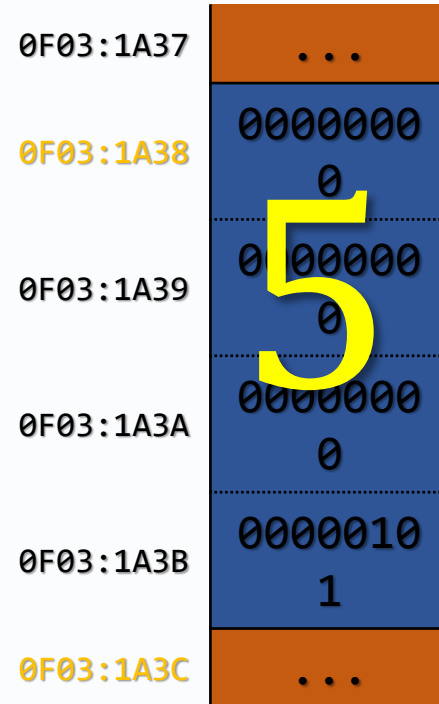
El dato (i) se accede a partir de su *dirección base* (0F03:1A38)

Dirección de la primera celda de memoria utilizada por el dato

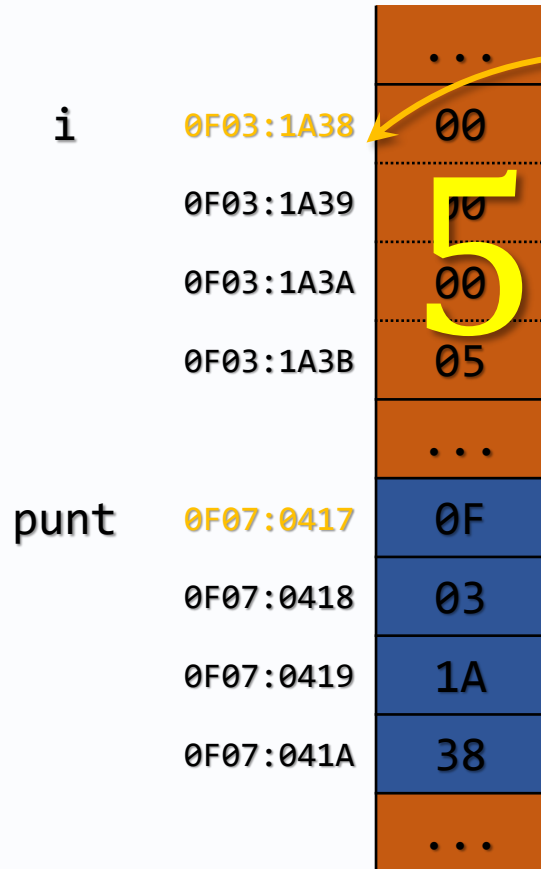
El tipo del dato (**int**) indica cuántos bytes (4) requiere el dato:

00000000 00000000 00000000 00000101 → 5

(La codificación de los datos puede ser diferente; y la de las direcciones también)

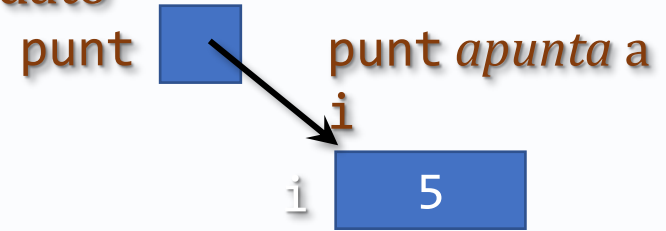


Variables punteros



Indirección:

Acceso indirecto a un dato



*¿De qué tipo es el dato apuntado?
¿Cuántas celdas ocupa?
¿Cómo se interpretan los 0/1?*

Punteros

DIRECCIONES DE MEMORIA

Un puntero es una variable que **almacena la dirección de memoria** de otra variable. Son el mecanismo subyacente de toda estructura dinámica en C++.

Operadores esenciales

```
int x = 10;
int* ptr = &x; // & = dirección de x
cout << ptr;   // imprime dirección (ej: 0x7ffd)
cout << *ptr;  // * = desreferencia → 10

*ptr = 25;     // modifica x a través del puntero
cout << x;     // imprime 25

// Puntero a struct
struct Nodo { int dato; Nodo* sig; };
Nodo* n = new Nodo();
n->dato = 5;    // operador flecha "->"
```

Relaciones clave

& – Address-of

Obtiene la dirección de memoria de una variable.

* – Dereference

Accede al valor almacenado en la dirección apuntada.

-> – Member Access

Accede a miembros de una estructura mediante puntero.



Declarar puntero

Asignar dirección

Desreferenciar

Estructuras (struct)

Una **struct** agrupa variables de distintos tipos bajo un mismo nombre, modelando entidades del mundo real. Son la base de los nodos en listas, árboles y grafos.

```
// Definición de struct Estudiante
struct Direccion {
    string ciudad;
    string pais;
};

struct Estudiante {
    int    carnet;        // identificador
    string nombre;        // nombre completo
    float  promedio;      // promedio académico
    Direccion domicilio; // struct anidada
};

// Uso
Estudiante e1;
e1.carnet   = 20240001;
e1.nombre  = "Ana López";
e1.promedio = 9.2f;
e1.domicilio.ciudad = "Bogotá";

// Con puntero dinámico
Estudiante* e2 = new Estudiante();
e2->nombre = "Carlos Ruiz";
delete e2;
```

Aplicaciones reales

Nodos de listas

Cada nodo es un struct con dato y puntero al siguiente.

Registros de BD

Una fila de tabla se mapea directamente a una struct.

Geometría 3D

Vértices, normales y colores empaquetados en una struct.

 Las estructuras anidadas permiten modelar jerarquías de datos complejas con claridad y reusabilidad.

Listas Enlazadas

Una lista enlazada es una colección de **nodos** donde cada uno contiene un dato y un puntero al siguiente. A diferencia de los arreglos, no requiere memoria contigua y crece o decrece dinámicamente.

```
struct Nodo {
    int    dato;
    Nodo* siguiente;
};

// Insertar al inicio O(1)
void insertarInicio(Nodo*& cabeza, int val) {
    Nodo* nuevo = new Nodo();
    nuevo->dato = val;
    nuevo->siguiente = cabeza;
    cabeza = nuevo;
}

// Eliminar nodo con valor buscado
void eliminar(Nodo*& cabeza, int val) {
    if (!cabeza) return;
    if (cabeza->dato == val) {
        Nodo* tmp = cabeza;
        cabeza = cabeza->siguiente;
        delete tmp; return;
    }
    Nodo* actual = cabeza;
    while (actual->siguiente &&
        actual->siguiente->dato != val)
        actual = actual->siguiente;
    if (actual->siguiente) {
        Nodo* tmp = actual->siguiente;
        actual->siguiente = tmp->siguiente;
        delete tmp;
    }
}
```

Lista vs Arreglo

Operación	Arreglo	Lista
Acceso por índice	$O(1)$	$O(n)$
Inserción al inicio	$O(n)$	$O(1)$
Eliminación	$O(n)$	$O(n)^*$
Memoria	Contigua	Dispersa
Tamaño	Fijo	Dinámico

* $O(1)$ si se tiene el puntero directo al nodo anterior.



Las listas son superiores cuando las inserciones/eliminaciones son frecuentes y el tamaño es impredecible.

Tipos Abstractos de Datos (TAD)

ABSTRACCIÓN

ENCAPSULAMIENTO

Un TAD define **qué operaciones** puede realizar una estructura de datos, sin especificar **cómo** se implementan internamente. Separa la interfaz del usuario de los detalles de implementación.



Pila (Stack)

LIFO — último en entrar, primero en salir.

Operaciones: `push`, `pop`, `top`. Uso: historial de navegador, recursión.



Cola (Queue)

FIFO — primero en entrar, primero en salir.

Operaciones: `enqueue`, `dequeue`, `front`. Uso: impresoras, scheduling de procesos.



Lista (List)

Secuencia ordenada con acceso posicional.

Operaciones: `insert`, `remove`, `get`. Uso: registros, colecciones ordenadas.

❏ El principio TAD es la base de la **Programación Orientada a Objetos**: define contratos de interfaz independientes de la implementación.

Matrices Dispersas por Tripletas

OPTIMIZACIÓN DE MEMORIA

Una matriz dispersa tiene la mayoría de sus elementos en **cero**. Almacenar todos los ceros desperdicia memoria. La representación por **tripletas (fila, columna, valor)** guarda únicamente los elementos no nulos.

Ejemplo: Matriz 5×5 con 4 elementos

Matriz original (25 celdas, 21 ceros desperdiciados):

	C0	C1	C2	C3	C4
F0	0	0	3	0	0
F1	0	0	0	0	0
F2	0	7	0	0	0
F3	0	0	0	9	0
F4	2	0	0	0	0

Representación tripleta

Fila	Columna	Valor
0	2	3
2	1	7
3	3	9
4	0	2

```
struct Tripletas {  
    int fila, col, val;  
};  
// Solo 4 registros vs 25 celdas  
// Ahorro: 84% de memoria
```

Matrices Dispersas con Listas Enlazadas

La representación con listas enlazadas supera a las tripletas al permitir **inserciones y eliminaciones dinámicas** sin reorganizar un arreglo. Cada elemento no nulo es un nodo que almacena su posición y valor.

```
struct NodoMatriz {
    int fila;        // índice de fila
    int columna;     // índice de columna
    double valor;    // valor no nulo
    NodoMatriz* siguiente; // enlace al próximo

    NodoMatriz(int f, int c, double v)
        : fila(f), col(c), valor(v),
          siguiente(nullptr) {}
};

class MatrizDispersaLista {
    NodoMatriz* cabeza;
public:
    MatrizDispersaLista() : cabeza(nullptr) {}

    void insertar(int f, int c, double v) {
        NodoMatriz* n = new NodoMatriz(f, c, v);
        n->siguiente = cabeza;
        cabeza = n;
    }

    double buscar(int f, int c) {
        NodoMatriz* actual = cabeza;
        while (actual) {
            if (actual->fila == f &&
                actual->col == c)
                return actual->valor;
            actual = actual->siguiente;
        }
        return 0.0; // elemento no encontrado
    }
};
```

Ventajas sobre tripletas

→ Inserción $O(1)$

No requiere desplazar elementos como en un arreglo ordenado.

→ Memoria exacta

Cada nodo ocupa solo lo necesario; no hay reserva previa.

→ Eliminación eficiente

Remover un elemento no nulo sin reconstruir estructuras.



El costo extra es el puntero adicional por nodo (~8 bytes en sistemas de 64 bits).

Matrices Dispersas Ortogonales

ESTRUCTURA AVANZADA

La representación ortogonal extiende la lista enlazada con **doble navegación**: cada nodo tiene un enlace al siguiente en su fila y otro al siguiente en su columna. Esto permite recorridos eficientes tanto horizontal como verticalmente.

```
struct NodoOrtogonal {
    int    fila, col;
    double valor;
    NodoOrtogonal* sigFila; // → siguiente en fila
    NodoOrtogonal* sigCol;  // ↓ siguiente en columna

    NodoOrtogonal(int f, int c, double v)
        : fila(f), col(c), valor(v),
          sigFila(nullptr), sigCol(nullptr) {}
};

class MatrizOrtogonal {
    vector<NodoOrtogonal*> cabFila; // cabezas por fila
    vector<NodoOrtogonal*> cabCol; // cabezas por col
    int filas, cols;
public:
    MatrizOrtogonal(int f, int c)
        : filas(f), cols(c),
          cabFila(f, nullptr),
          cabCol(c, nullptr) {}

    void insertar(int f, int c, double v);
    // inserta manteniendo orden en ambas listas
};
```

Estructura de nodos



Recorrido por fila

Seguir sigFila desde cabFila[i]

Recorrido por columna

Seguir sigCol desde cabCol[j]

Operaciones con Matrices Dispersas

Las operaciones fundamentales deben aprovechar la estructura dispersa para evitar procesar los ceros, logrando eficiencia significativa frente a matrices densas.

1

Inserción

Crear nuevo nodo (f, c, v) y enlazarlo en la posición correcta de la lista o actualizar el valor si ya existe. Complejidad: **$O(\text{nnz})$** donde nnz = elementos no nulos.

2

Búsqueda

Recorrer la lista comparando (fila, columna). Si no se encuentra, el valor es implícitamente **0**. Ortogonal mejora esto a $O(\text{nnz_fila})$.

3

Suma $A + B$

Combinar las dos listas: si ambas tienen elemento en (i,j), sumar valores; si solo una lo tiene, copiar directamente. Solo se procesan elementos no nulos.

4

Multiplicación $A \times B$

Para cada fila de A y columna de B, sumar productos de elementos coincidentes. La dispersión reduce drásticamente las multiplicaciones efectivas.

5

Transposición A^T

Intercambiar (fila, columna) de cada nodo. En ortogonal: intercambiar los punteros sigFila y sigCol, y rotar los arreglos de cabezas.

Aplicaciones Reales

Las matrices dispersas y estructuras dinámicas son la infraestructura invisible detrás de las tecnologías más usadas del mundo.



Inteligencia Artificial

Redes neuronales con millones de pesos, la mayoría cercanos a cero tras el entrenamiento. Librerías como **TensorFlow** usan sparse tensors.



Motores de Búsqueda

El índice invertido de Google es una matriz dispersa documento-término. Permite buscar en miles de millones de páginas en milisegundos.



Redes Sociales

La matriz de adyacencia de Facebook (3B usuarios) es >99.9% cero. Solo se almacenan las amistades reales con listas de adyacencia.



Sistemas GIS

Mapas de carreteras y redes de transporte modelados como grafos dispersos para calcular rutas óptimas (algoritmo de Dijkstra).



Comparativa General de Estructuras

Seleccionar la estructura correcta es una decisión de ingeniería que impacta directamente el rendimiento y la escalabilidad del sistema.

Característica	Arreglo	Lista enlazada	Tripletas	Lista dispersa	Ortogonal
Memoria usada	$O(n^2)$	$O(n)$	$O(nnz)$	$O(nnz)$	$O(nnz + n)$
Acceso por posición	$O(1)$	$O(n)$	$O(nnz)$	$O(nnz)$	$O(nnz/n)$
Inserción dinámica	No	Sí $O(1)$	Limitada	Sí $O(1)$	Sí $O(nnz/n)$
Recorrido por fila	$O(n)$	$O(nnz)$	$O(nnz)$	$O(nnz)$	$O(nnz_fila)$
Recorrido por columna	$O(n)$	$O(nnz)$	$O(nnz)$	$O(nnz)$	$O(nnz_col)$
Implementación	Simple	Media	Fácil	Media	Compleja

 **nnz** = número de elementos no nulos. Para matrices con densidad <5%, las estructuras dispersas son siempre superiores en memoria y tiempo.

Caso de Estudio: Red de Transporte Urbano

PROBLEMA REAL

SOLUCIÓN CON MATRICES DISPERSAS

Una ciudad con **500 estaciones de metro** tiene una matriz de adyacencia de $500 \times 500 = 250,000$ celdas. Con un promedio de 3 conexiones por estación, solo 1,500 son no nulas — un **99.4% de desperdicio** con arreglos densos.

```
// Red de transporte como matriz ortogonal
// Nodo representa conexión entre estaciones
struct Conexion {
    int     estOrigen, estDestino;
    double distancia_km;
    double tiempo_min;
    Conexion* sigRuta;    // próxima desde origen
    Conexion* sigDestino; // próxima hacia destino
};

class RedMetro {
    vector<Conexion*> porOrigen; // rutas desde E_i
    vector<Conexion*> porDestino; // rutas hacia E_j
    int numEstaciones;

public:
    RedMetro(int n) : numEstaciones(n),
        porOrigen(n, nullptr),
        porDestino(n, nullptr) {}

    // Agregar ruta bidireccional
    void agregarRuta(int o, int d,
        double km, double min) {
        Conexion* c = new Conexion{o,d,km,min};
        c->sigRuta    = porOrigen[o];
        porOrigen[o] = c;
        // también registrar en destino...
    }

    // Ruta más corta: Dijkstra  $O((V+E) \log V)$ 
    vector<int> rutaMinima(int origen, int meta);
};
```

Resultados del análisis

99.4%

Reducción de memoria

De 250,000 celdas a solo 1,500 nodos activos.

$O(E \log V)$

Dijkstra eficiente

Solo procesa aristas existentes, no las 248,500 vacías.



Este mismo modelo se usa en Google Maps, Waze y los sistemas de navegación de Amazon para calcular rutas en tiempo real.

Actividad Interactiva – Discusión en Clase

1

Memoria y punteros

¿Qué ocurre si se llama a `delete` dos veces sobre el mismo puntero? ¿Cómo lo evitarías en producción?

2

Listas vs arreglos

Tu app almacena el historial de reproducción de un usuario (inserciones frecuentes al inicio). ¿Usarías lista o arreglo? Justifica con complejidades.

3

TAD y abstracción

¿Por qué la STL de C++ implementa `std::stack` usando internamente un deque? ¿Podría usarse una lista? ¿Qué cambia para el usuario?

4

Matrices dispersas

Una matriz $10,000 \times 10,000$ tiene 500 elementos no nulos. Calcula el porcentaje de dispersión y elige justificadamente entre tripletas y representación ortogonal.



Ejercicios de Programación en C++

5 RETOS PROGRESIVOS

1 Gestión de memoria básica

Crear un arreglo dinámico de n enteros, llenarlo con cuadrados perfectos (1, 4, 9...), imprimirlo en orden inverso y liberar la memoria. Verificar con Valgrind que no hay leaks.

2 Lista enlazada de estudiantes

Implementar una lista enlazada de `struct Estudiante` con carnet, nombre y promedio. Funciones: insertar ordenado por carnet, eliminar por carnet, imprimir todos.

3 Pila con TAD

Diseñar el TAD `Pila` usando lista enlazada internamente. Implementar `push`, `pop`, `top`, `isEmpty`. Usar la pila para invertir una cadena de texto.

4 Matriz dispersa por tripletas

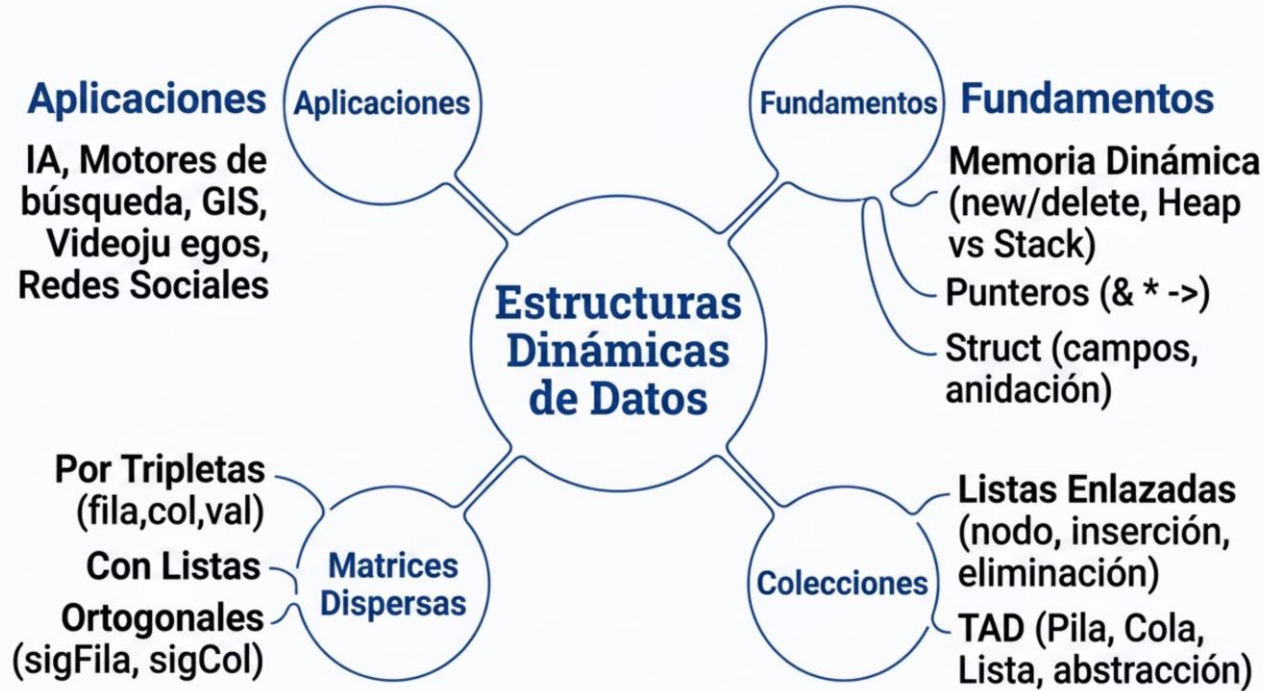
Leer una matriz 8×8 desde archivo, construir su representación por tripletas y calcular su transpuesta sin usar memoria adicional para ceros.

5 Red de conexiones ortogonal

Implementar una `MatrizOrtogonal` para modelar 6 ciudades y sus conexiones. Imprimir todas las ciudades accesibles desde la ciudad 0 recorriendo la lista de filas.

Resumen – Mapa Conceptual

Todos los temas del curso convergen en un único objetivo: **gestionar la memoria de forma eficiente y flexible** para construir software escalable y de alto rendimiento.



Dominar estos fundamentos te prepara para cursos avanzados de Algoritmos, Bases de Datos, Compiladores e Inteligencia Artificial.